Universitat Pompeu Fabra Barcelona

Pau Reig Vaqué - u215101
Marc Rodríguez Vitolo - u215105
Clara Serra Borràs - u204635

REPORT
Part 4: RAG, User Interface, and Web Analytics

1. User Interface

In this part of the project, we focused on implementing the user interface and connecting it to the search engine. We began by replacing the original dataset, fashion_products_dataset.json, with cleaned_fashion_products.zip, which contains the processed and cleaned columns from our previous assignments. These cleaned fields allow us to initialize the Flask application much faster, as no additional preprocessing is required. After this change, we obtain the corpus variable enriched with more information (tokenized fields) than in the original dataset.

The first two requirements of this section ask us to create a main web page with a central search box for users to enter a query and a button to execute the search, and to ensure that when the button is clicked, the search text is passed to the engine's search function. These features were already implemented; however, as we explain later, we also added a selection box that allows users to choose one of the three available search engines before clicking the search button.

The third requirement consisted of creating a general search function and introducing all the necessary parameters so that it could later be used to obtain the ranked documents. Since we implemented the three search engines from the previous assignments, TF-IDF, BM25, and our own custom algorithm, which we named Pepito, we needed to include a large number of parameters: the raw query (search_query), the processed query (query_terms), the dataset in corpus format (corpus), which is required to return documents in JSON form, the dataset in DataFrame format (corpus_dataframe), which is needed due to the way our algorithms are implemented, the variables required for ranking the documents (index, tf, idf, and bm25), and finally the search engine selected by the user (selected_engine).

The next requirement was to implement the three algorithms using this general function. Thus, the general search function calls another function that performs the search in the corpus and behaves according to the selected search engine. For each search engine, we reused the implementation from the previous assignments. The TF-IDF engine calls search_tf_idf, which then calls rank_documents. The BM25 engine uses the get_top_n function, and the Pepito engine uses search_pepito, which calls rank_documents_pepito.

To comply with the guideline of real use case optimization: "optimize your algorithms for retrieving the best results, faster, cleaner, and better suited to the user's information needs", we precomputed the index, tf, df, idf, and BM25 model at Flask initialization, so these computations do not need to be repeated for every query. This is also why these elements are passed as parameters to the search function. For the same reason, we replaced the original dataset with the already processed one, which allows for significantly faster execution. Moreover, we have modified the BM25 algorithm to leverage the inverted index directly, avoiding iteration over the entire corpus. By filtering only the documents that contain the query terms and computing scores exclusively for them, the search becomes much more efficient, especially on large datasets, while maintaining accurate ranking results.

Regarding the requirements for the results page, we use the documents returned by the general search algorithm and display them in an HTML template. We decided to present all the elements specified in the assignment. First, the documents are shown in the order determined by the ranking. For each document, we include the required fields: Title , Description, Selling price and discount, Average rating, URL (which links to the original product page). This ensures that each result record properly represents a document from the corpus and provides users with the essential information they need to evaluate the relevance of each item. We have made some changes to the HTML to obtain more attractive and visual results to the user. We believe that this improvement could bring benefits by enhancing the overall user experience, making the interface more intuitive, and enabling quicker evaluation of product relevance.

The sixth requirement was to create an LLM-generated summary on the results page. The project already included a basic implementation of this functionality, so we focused on improving and extending it. These enhancements are explained in detail in Section 2 of this report.

Regarding the document details page, we updated the corresponding HTML template (doc_details) so that it receives the selected document in its corpus format. This page is intended to display the full set of information available for each product. With this objective in mind, we have designed the page to present additional relevant attributes from the dataset beyond those shown on the results page. The information shown includes the product title, category and sub-category, full description, and brand, along with all available product details extracted from the dataset. On the right side of the page, we display the selling price, discount, average rating, and the stock status highlighted with color. Additionally, the page presents all associated product images in an image gallery. This ensures that users can view the complete set of attributes for each item in a clear , visual and structured format.

2. RAG

In this part of the project we have improved the Retrieval Augmentation Generation (RAG) component. The original implementation provided only minimal metadata to the language model and relied heavily on the model's ability to infer product quality without structured context. As a consequence, the generated responses tended to be generic.

First, we have identified that there was an extremely limited amount of information passed to the LLM. The original implementation only included a list of product titles and IDs. Therefore, we thought that the model does not have enough information to determine whether a product was truly suitable for the user's request. To address this, we have created a compact but informative metadata representation that extracts key attributes such as brand, category, selling and actual prices, discount, color, fabric, stock status, rating, and a truncated product description. We have decided to truncate the description to reduce the number of tokens, because otherwise the LLM could produce a bad answer due to the amount of information provided.

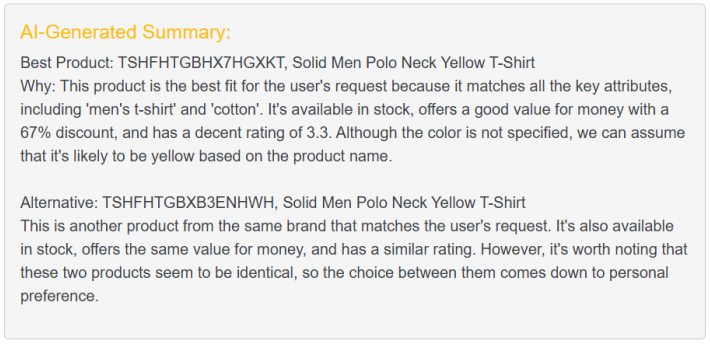
A second weakness was the absence of any pre-evaluation of the retrieved products. The original system always produced a recommendation. Although it could be mentioned in the prompt, it doesn't take it into account. This can mislead users and make the system appear

unreliable. To avoid this, we have introduced a pre-filtering mechanism that detects cases where none of the retrieved products should reasonably be recommended. The filter checks whether the results list is empty, whether all products are out of stock, whether a user-specified budget is entirely unmet, and whether all rated products fall below a minimum quality threshold. To create this prefilter, we have made some assumptions, for example, that products with ratings below 3 are typically unreliable. We have also implemented the budget limit prefilter, but this doesn't work in our implementation since we consider that documents are ranked considering all the words. Therefore, if the query is "jacket under 100" this will not return any document since "under" is not normally in the information of the document. So, we kept this prefilter to improve the ranked documents in the future with that kind of considerations. The pre-filtering step allows the system to return "there are no good products that fit the request", instead of forcing the language model to improvise.

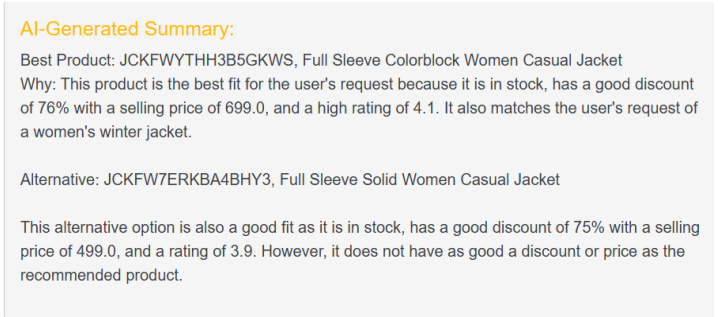
Finally, we have changed the structure and clarity of the prompt, to make it more adequate with respect to our modifications. The new prompt directly guides the model to compare products on the dimensions users commonly care about, such as relevance to the query, stock status, value for money, and key attributes. The prompt explicitly instructs the model to identify one best product, justify the selection using factual metadata, optionally suggest an alternative, and strictly avoid hallucinating when no product meets the criteria.

Together, these improvements produce a system that is more aligned with user intent, more transparent in its reasoning, and more resistant to hallucinations. However, we could make other optimizations in the future, such as query rewriting and expansion, to better understand what the user is interested in.

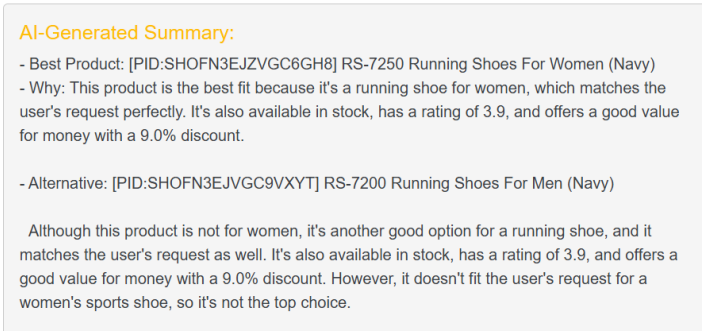
In the following screenshots, we show the RAG responses for the five queries we have used throughout the project.



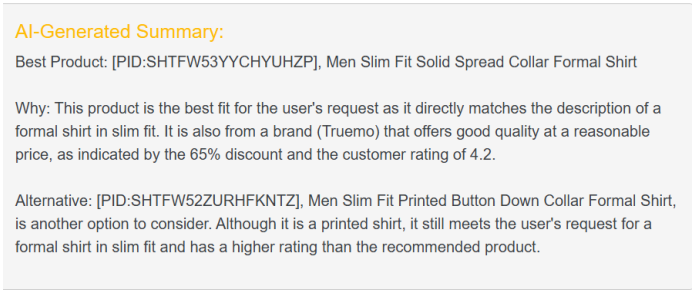
Query: cotton t-shirt men



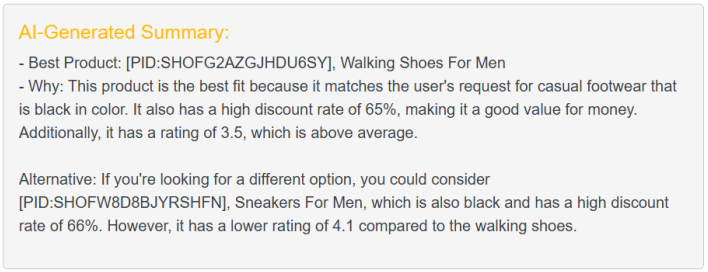
Query: women winter jacket



Query: sports shoes running



Query: formal shirt slim fit



Query: casual footwear black

The results across the five queries show that the improved RAG module produces more precise and context-aware recommendations. The system consistently selects products that match the user intent while clearly justifying the choice. We also observe, in other queries, that the pre-filtering mechanism prevents unsuitable suggestions, leading to more reliable outputs. Overall, we think that the generated answers are more structured and aligned with user needs.

3. Web Analytics

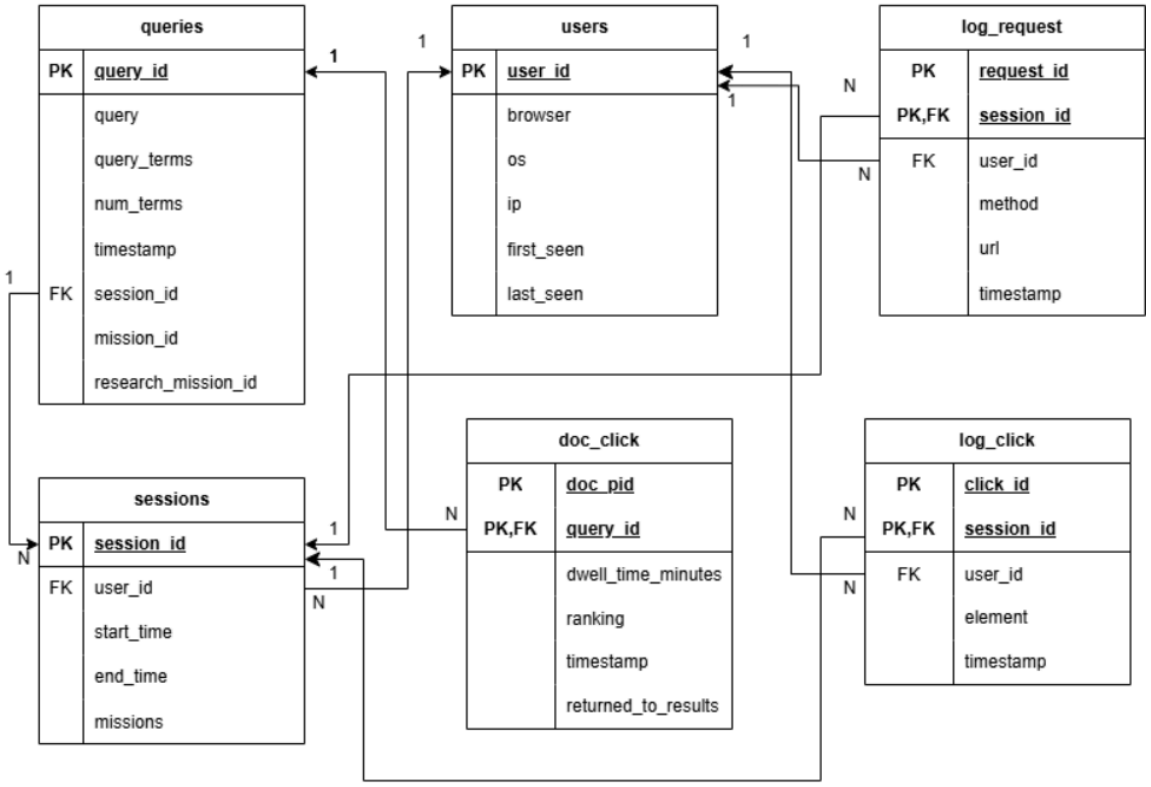
In this part of the project we have implemented web analytics, based on a robust mechanism for effectively tracking and analyzing how people use our website and how the search engine is used.

Our implementation consists of two steps: data selection and collection, where we have analyzed which data was interesting to store; and data storage, where we have thought about and designed a data model to store the information that we collect about website usage from the previous point.

In order to do that and to store data for long-term persistence, we have decided to use https://railway.com/ . This tool is a complete cloud hosting platform, and we have used it to

store the data permanently by taking advantage of its free first-month option. Therefore, in the web we have created the tables with the primary keys and the necessary attributes to store the data and structure it as a relational model. To do this, we have created a new .py file in the analytics folder called database.py, where we can find the functions needed to perform inserts and gets; that is, to insert newly generated data and retrieve it whenever needed to display it in charts (dashboards).

Thus, returning to the data-selection stage, we have created the following tables in Railway with the information/attributes shown in the relational model below.



First, we have created a set of tables to store information related to HTTP requests and basic navigation activity. These include data about each request, general clicks, and HTTP session information, which is essential for identifying when a visit begins and ends. To support this, we have defined three tables: log_request, log_click and sessions, each responsible for capturing a different aspect of user interaction.

The log_request table stores detailed information about every HTTP request received by the server. Its attributes include the request_id and session_id, as well as user_id associated with the request. Additionally, the table logs the HTTP method used (such as GET or POST), the requested URL, and the timestamp at which the request occurred. These fields allow us to reconstruct the user's navigation path and detect patterns.

Similarly, the log_click table records all general click events on the website that are not specific to document results. Each click event includes a click_id and the session_id it belongs to, along with the user_id, the interface element that was clicked, and the

timestamp. This information is useful for understanding how users interact with the interface and which elements attract the most attention.

Finally, the sessions table groups all user activities into time-based sessions. Each session entry stores a unique session_id, the user_id who initiated it, and the start_time and end_time, allowing us to identify the length and structure of each visit. The table also includes a missions field, which references the number of logical missions that occurred within that physical session.

Next, we have addressed the queries. We have defined a table with an auto-incrementing identifier (serial) and attributes such as the original query, the processed query, the number of resulting terms, the number of returned documents, and the exact timestamp of the search. Additionally, each query is associated with a session (the user's physical session), a mission (an identifier of queries with the same goal in the same session/ one user) and research mission (an identifier of queries with the same goal of all the sessions of one user). All these three identifiers are unique to avoid conflicts across different users.

Another important aspect was storing information related to document clicks. Each click record contains the corresponding query_id, the product ID of the consulted document, its ranking position, the dwell_time_minutes (obtained through the log_dwell_time() function), the timestamp of the action, and whether the user returned to the results page after visiting the document.

Finally, we have created a table to manage user information. For each user, we store an identifier (serial) along with the browser used, operating system, IP address, and access date. This information is key for detecting usage patterns, preferences, and potential differences across devices.

Everything is stored in the external database so we have the information of every user that executes the website. In local we have only kept a copy of the sessions and the queries corresponding to the user that is executing the web application. The information is used to compute the missions when new queries are added. For the other usages, like the counter of clicks when you open a document, it is obtained by executing queries in the external database. This way we have avoided using a lot of memory to store all the movements user's do.

The standard procedure of interaction is the following:

When the web application is executed, we have checked in the cookie if that user has already executed the website previously. If he does not, it creates a new user in the database with the corresponding information, when inserting the user we get the id that will be assigned to that user. If he has previously executed the website we have requested the information corresponding to that user.

The doclicks, log requests, log clicks are inserted in the external database when they are created. When needed for the stats, dashboard and others we launch queries to the database so we get the information we need to show.

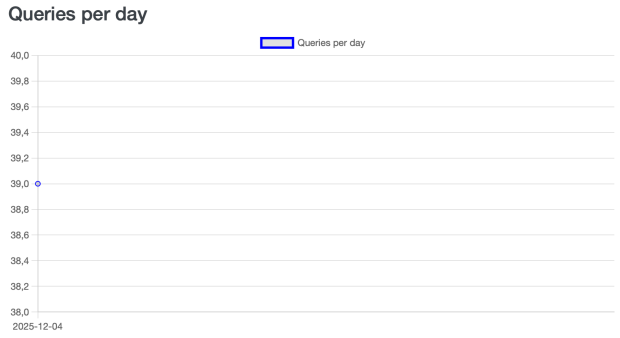
Universitat Pompeu Fabra Barcelona

Pau Reig Vaqué - u215101
Marc Rodríguez Vitolo - u215105
Clara Serra Borràs - u204635

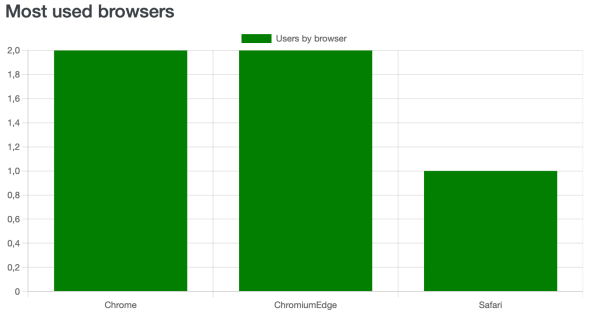
As we explained earlier, we have implemented queries to retrieve the content from the database. Then, first of all, we have modified the stats page using the database information so we could display data we have considered relevant. In addition to the clicked documents, which was already implemented, we can now see the total number of users, sessions, queries, total document clicks, and the average dwell time. We believe this is important because it allows us to demonstrate that our analytics system goes beyond simple click tracking, so we provide a clearer picture of how people interact with the search engine.

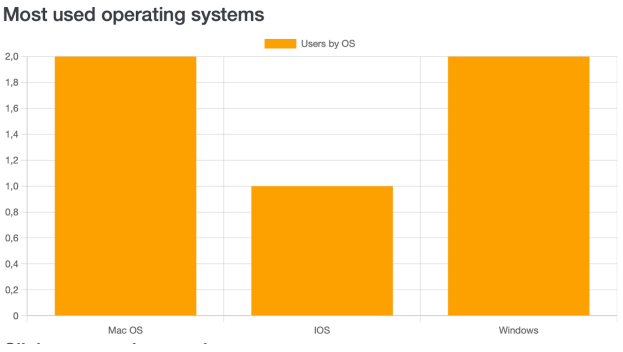
Moreover, we have created some dashboard to visualize how users interact with our website and search engine. Once the relevant data is selected and collected with the specific queries, the dashboard page presents this information through a set of visualisations that illustrate real usage patterns. In the following images we can see the dashboard from our web.

The first graph shows the total number of queries made since the app was launched. It illustrates the evolution of queries per day. As we finalized the app on December 4th, we only see a single data point, because the table was modified several times and we were unable to record queries across different days. This graph is useful for assessing whether users are actively engaging with our application.



These graphs show the proportion of users according to the browser or operating system they use to access the website. The main goal is to understand the environment from which they use the system, which is key both for interface design and for detecting compatibility issues. We can see that we have a different range of users across browsers and operating systems, and this helps us identify the dominant platforms. This information is especially useful for prioritizing optimization.

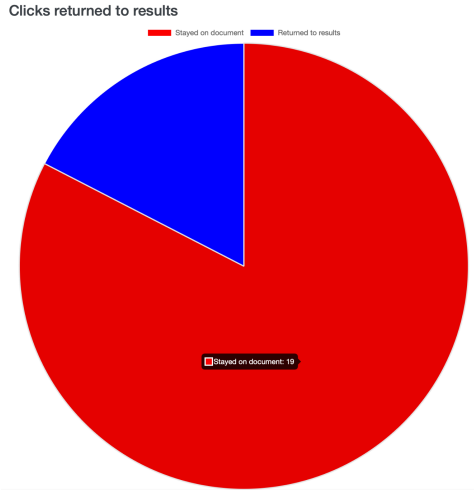




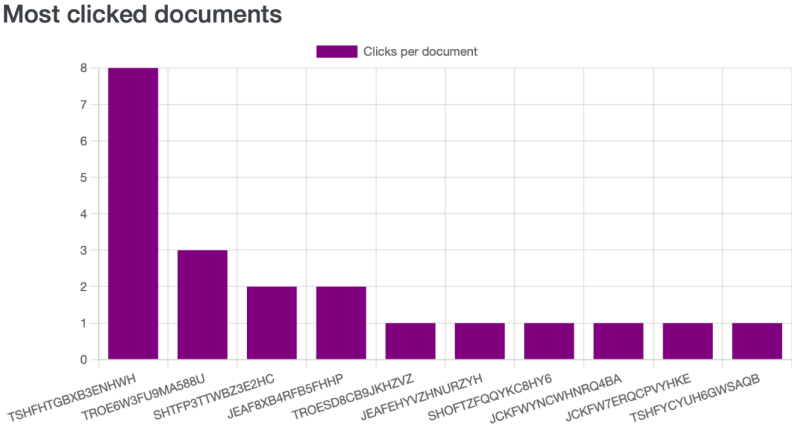
Universitat Pompeu Fabra Barcelona

Pau Reig Vaqué - u215101
Marc Rodríguez Vitolo - u215105
Clara Serra Borràs - u204635

The following graph shows the proportion and number of users who either stayed on the document or returned to the results page. This visualization helps illustrate whether users were satisfied with the document they accessed. As we can see, the majority of users chose to remain on the document.



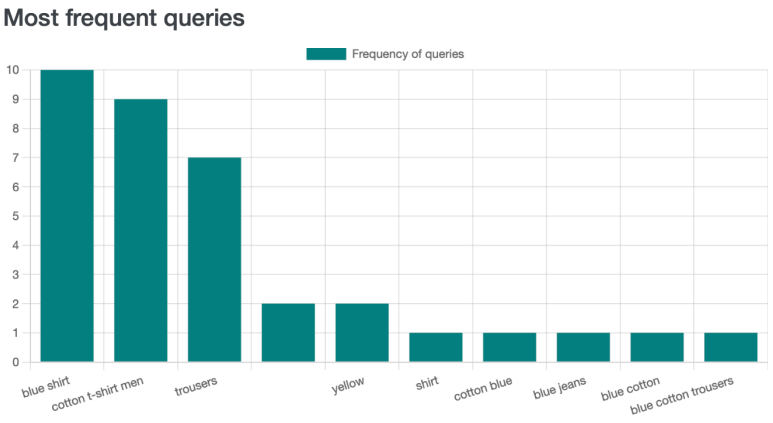
The following graph shows which documents have received the highest number of clicks from users, using the data collected in the doclicks table. The goal is to identify which products generate the most real interest after being displayed in the search results. Documents with more clicks indicate that users consider them relevant or appealing, while those with fewer interactions may suggest that they do not meet user expectations as well or that they appear too far down in the ranking.



The last graph shows the most frequent queries made by users. This visualization highlights which types of searches appear most often, allowing us to detect common interests. By identifying these recurring patterns, we gain insight into real user needs and can assess whether our search engine effectively supports them. This information is also useful for guiding future improvements.

Universitat Pompeu Fabra Barcelona

Pau Reig Vaqué - u215101
Marc Rodríguez Vitolo - u215105
Clara Serra Borràs - u204635



GitHub URL: https://github.com/mmarcrv/irwa-search-engine-G005
TAG: IRWA-2025-part-4
AI Usage: We used ChatGPT and Gemini to review our draft explanation of pre-processing decisions and to help us to make some html code and database queries.